BOB-137 â€” merge service (7187) wedge: captured forensics

Revision: 3 **Last modified:** 2026-08-20T16:10:00Z

Captured 2026-08-20 on the live stack. Evidence class: runtime (Â§11.4.226) â€” live process, socket and thread state plus in-process stack dumps, not source inspection.

STATUS: ROOT CAUSE ESTABLISHED. The spinning frame is named in Â§7 and proven by seven all-thread stack dumps taken while the defect was active (`stall_dump_1.txt`). Two inherited premises from Revision 2 are REFUTED under control conditions in Â§6 â€” read that section before citing Rev-2.

No fix is proposed or applied here (Â§11.4.102 Iron Law). The only code change is an observe-only instrument (Â§9).

---

1. The wedge, measured (Revision 1 â€” unchanged, still accurate)

Container `qbittorrent-proxy`, up 3h53m, reported `Up 4 hours (healthy)`:

```
curl --max-time 6 http://localhost:7186/       -> HTTP 200 in 0.096s
curl --max-time 6 http://localhost:7187/       -> HTTP 000 after 6.004s
```

Both ports are served by the SAME process â€” `ss -ltnp` shows one pid holding fd 3 (7186) and fd 7 (7187). Not a crashed worker: one loop inside a live process stopped servicing requests while another in the same process kept working. See `sockets.txt`, `threads.txt`, `container_log_tail.txt`.

2. Reproduced deterministically (NEW, Revision 3)

`scripts/diagnostics/bob137_soak.sh 2400 6` drives sustained concurrent `POST /api/v1/search` fan-out while an independent prober samples both ports every 5s. Measured 2026-08-20T15:39Zâ€“15:46Z:

```
probes total: 26
7187 outages: 22      <- merge service dead
7186 outages:  0      <- download proxy healthy throughout
```

The asymmetry is total: **22 of 26 probes found 7187 unreachable and 0 of 26 found 7186 unreachable**, in the same process, at the same instants.

The reproduced wedge carries the SAME socket fingerprint as the originally reported one (`repro_socket_state.txt`), which is what binds this repro to the reported defect rather than to a look-alike (Â§11.4.199, precondition provenance = **observed**, not constructed):

Signal	Originally reported (Â§1)	Reproduced (Â§2)
7187 LISTEN accept backlog	Recv-Q 6	Recv-Q 88
7187 conns in CLOSE-WAIT with unread bytes	162/162/162/79/1/1/1	85/168/194/220/201/â€¦
7186 LISTEN backlog	0	0
fd exhaustion	no (48/16384)	no (118)
thread count	16	19

3. Why 7186 survives while 7187 dies

They are different servers on different threads of one process:

- **7186** is `download_proxy.run_server()` â€” a blocking `socketserver` on its own thread (`main.py:242`). CPython releases the GIL every switch interval (default 5 ms), so it keeps getting scheduled even while another thread runs Python continuously.
- **7187** is uvicorn on a single asyncio event loop (`main.py:288`). It has no second thread to fall back on: if any callback on that loop does not return, **no** other callback runs â€” not accept, not read, not the response write.

That is why the accept queue backs up (Recv-Q 88 on the LISTEN socket) and why established connections sit in CLOSE-WAIT with the clientâ€™s request bytes still unread: the kernel completed the handshakes and buffered the bytes, the client gave up and sent FIN, and the application never ran to read or close either.

4. Log silence and self-clearing, explained â€” and measured

Both follow from Â§3. Nothing is logged because nothing on the loop progresses; the service recovers with no restart because the callback eventually returns.

Revision 3 measured two complete episodes end to end, with NO restart and no intervention (watchdog log):

```
15:40:03  SILENT for 22s (episode 1) -- dumping all thread stacks
15:49:18  RECOVERED (episode 1)          -> ~9 min 37 s of event-loop sile
15:49:38  SILENT for 21s (episode 2) -- dumping all thread stacks
15:55:13  RECOVERED (episode 2)          -> ~5 min 56 s of event-loop sile
```

Load was stopped at 15:44Z, yet the loop kept spinning until 15:55Z â€” **eleven minutes of stall after the last request was sent**, draining merges already queued. Longest single stretch of continuous silence recorded by the watchdog: `loop_silent_for=572.7s`. This is the mechanism behind the originally reported ~2 h of log silence: the stall outlives the traffic that caused it, and concurrent searches queue their merges behind one another.

Immediately after recovery the service answered normally (`HTTP 200 in 3.05s`, then sub-second), with no restart â€” which is exactly the Revision 2 Â§6 observation, now explained rather than merely noted.

Episode 3: it is NOT soak-only

At 15:56:17Z a THIRD episode began with **zero load from this investigation** (verified by enumerating real pids and excluding the probing shellâ€™s own command line â€” the Â§11.4.196(D)/Â§12.12 `pgrep -f` self-match footgun, which did initially produce a false hit here). Its dump shows the identical path:

```
deduplicator.py, line 446 in _detect_content_type
deduplicator.py, line 201 in _extract_identity_from_result
deduplicator.py, line 224 in _check_match
deduplicator.py, line  81 in merge_results
search.py,       line 914 in _run_search
routes.py,       line 447 in _background
```

So the defect fires under ordinary traffic reaching `POST /api/v1/search`, not only under the synthetic soak. Any search large enough to make `merge_results` run long will stall the merge service for other callers.

5. SIGTERM slowness is a SEPARATE, benign issue (NEW â€” decoupled)

`podman restart` reported `StopSignal SIGTERM failed to stop container in 10 seconds, resorting to SIGKILL` on a **healthy, freshly-started** container. It is therefore NOT a symptom of the wedge. Cause is in `main()`: the handler sets `_shutdown_event`, then `main()` does `proxy_thread.join(timeout=5)` + `fastapi_thread.join(timeout=5)` on two threads that never exit â€” 10 s total, which equals podmanâ€™s stop grace period. Tracked separately; not part of this root cause.

6. REFUTED inherited premises (Â§11.4.205(4), Â§11.4.201)

Revision 2 diagnosed â€œGIL starvationâ€ from two readings. Both are refuted:

(a) â€œ1 thread in state R with `wchan=0`â€ is the OBSERVER ITSELF. Demonstrated under control conditions on this host: a thread that

reads `/proc/self/task` is by definition running while it reads, so it reports `state=R`, `wchan=0` â€” and `d_utime=0`, i.e.Â it consumed no CPU:

```
observer tid: 312722
  tid=312721 state=R wchan=futex_wait_queue
  tid=312722 state=R wchan=0 <== THE OBSERVER ITSELF
```

The same `R/wchan=0/d_utime=0` row appears in the self-testâ€™s *golden-bad* case where the loop is blocked in `time.sleep` and **nothing is spinning**. Observer-induced signal (Â§11.4.201(10)); it is not evidence of a spinner.

(b) `ps %CPU 20.6` is a LIFETIME AVERAGE, not an instantaneous rate. `%CPU` is total CPU time Ã· elapsed time since process start. Over 3h53m it says roughly 48 minutes of CPU were consumed at some point; it says nothing about what the process was doing at the moment of measurement (Â§11.4.201(9), field identity and semantic class â€” a cumulative ratio read as an occupancy).

(c) â€œ14 threads in `futex_wait_queue`â€ is the normal idle state of any condition-variable waiter â€” here the `anyio` worker threads and the default `ThreadPoolExecutor` (visible by name in `stall_dump_1.txt`). Not a signature.

The Rev-2 conclusion (â€œa thread busy-looping holds the GIL and everything else queues on itâ€) was directionally close to the truth but rested on three non-load-bearing readings. The actual mechanism (Â§7) was established from per-thread CPU deltas and in-process stack dumps instead.

7. ROOT CAUSE â€” the spinning frame

`Deduplicator.merge_results()` performs an O(NÂ²) CPU-bound, regex-heavy deduplication synchronously on the asyncio event-loop thread.

Path: `routes.py:447 _background` (an asyncio task) â†’ `await orch._run_search` â†’ `search.py:914` calls `self.deduplicator.merge_results(all_results)` as a plain synchronous call. It is never handed to an executor, so for its entire duration the merge serviceâ€™s only event loop runs no other callback.

Seven dumps across one continuous 5.5-minute stall (`stall_dump_1.txt`):

```
===== BOB-137 STALL DUMP #1 loop_silent_for=21.7s  loop_tid=11 =====
  File ".../re/__init__.py", line 186 in sub
  File ".../merge_service/deduplicator.py", line 294 in _normalize_name
  File ".../merge_service/deduplicator.py", line 357 in _compare_identitie
  File ".../merge_service/deduplicator.py", line 226 in _check_match
  File ".../merge_service/deduplicator.py", line  81 in merge_results
  File ".../merge_service/search.py",       line 914 in _run_search
  File ".../api/routes.py",                 line 447 in _background
  File ".../asyncio/events.py",             line  88 in _run          <--
  File ".../asyncio/base_events.py",        line 1999 in _run_once
```

Per-thread CPU delta, sampled over 1.0 s inside each dump:

```
DUMP #1 loop_silent_for= 21.7s   tid=11 state=R d_utime=97 d_stime=1  <--
DUMP #2 loop_silent_for= 82.9s   tid=11 state=R d_utime=97 d_stime=1  <--
DUMP #3 loop_silent_for=144.1s   tid=11 state=R d_utime=98 d_stime=1  <--
DUMP #4 loop_silent_for=205.3s   tid=11 state=R d_utime=81 d_stime=1  <--
DUMP #5 loop_silent_for=266.5s   tid=11 state=R d_utime=90 d_stime=1  <--
```

Every other thread in every dump: `d_utime=0`. The loop thread alone burns 81â€“98 ticks per second â€” 81â€“98 % of one core, in **user** time, continuously.

It is executing, not blocked. The outer frames are identical in all seven dumps (`merge_results` â†’ `_check_match`) while the inner frame MOVES between them â€” `_normalize_name`/`re.sub` (line 294), `_detect_content_type`/`re.search` (474), `_extract_identity_from_result`/`re.search` (209), `_compare_identities` (357, 358). A stationary outer frame with a moving inner frame and sustained user-space CPU is a CPU-bound loop, and is exactly what distinguishes it from a blocking syscall (which would show a fixed frame and `d_utime=0`).

Why it grows super-linearly and is expensive per step

`merge_results` (deduplicator.py:73-95) pops a seed and compares it against every remaining candidate â€” N(N-1)/2 comparisons in the worst case, when few results match. Measured growth on synthetic data is super-linear but not yet fully quadratic at N<=400; see Â§8 for the numbers and the honest boundary.

Each comparison re-derives everything from scratch; nothing is cached (`grep -n "lru_cache" merge_service/deduplicator.py` â†’ no matches):

- `_check_match` calls `_extract_identity_from_result` **twice** â€” once for the seed and once for the candidate â€” so the seedâ€™s identity is recomputed for every one of its candidates. Each call runs a season/episode regex, `_detect_content_type` (a chain of up to ~15 sequential `re.search` calls), plus resolution and codec regexes.
- `_compare_identities` then calls `_normalize_name` on both titles; each call runs 5 `re.sub` passes.

The module contains 30 `re.*` call sites on this path. Order ~25 regex operations per comparison Ã— NÂ²/2 comparisons â‰ˆ O(10 NÂ²) regex operations. Single trackers were observed returning up to 189 results (`container_log_tail.txt`); summed across the 43 managed plugins, N in the low thousands puts this in the 10â·â€“10â¸ regex-operation range for ONE search.

Why stalls reach hours

Concurrent searches do not overlap â€” each queues its own `merge_results` on the same single loop, so their costs are **additive**. Observed here: load

was stopped at 15:44Z and the loop was still spinning at 15:46Z, draining merges already queued. The originally reported ~2 h of log silence is consistent with several large fan-outs queued behind one another.

The last lines before the original silence (`container_log_tail.txt`) are a security-suite fan-out across ~43 trackers with XSS/SQLi payload queries, followed by ~11 repeats of â€œLoaded configuration for qBittorrentâ€¦ â€” the memoised `get_config()` logging once per concurrent caller that raced its `_config is None` check. That is the concurrency burst which fed the merges.

Every observation, accounted for

Observation	Explained by
7187 dead, 7186 alive, same process	Â§3 â€” one loop vs a separate thread
CLOSE-WAIT with unread request bytes	loop never runs accept/read callbacks
7187 LISTEN accept backlog (Recv-Q 6 â† 88)	same
container log silent for ~2 h	nothing on the loop progresses
self-clears with no restart	the merge finally returns
16 threads wedged vs 5 idle	fan-outâ€™s anyio + executor pool
high cumulative process CPU	~1 core of regex work per merge
stall duration unbounded/variable	O(NÂ²) in results, additive across searches

8. Scale, measured (contention-immune)

A first attempt to TIME `merge_results` ran inside the wedged container and was contaminated by CPU contention with the live spin (N=100 took 8.5 s while N=200 took 3.7 s â€” an ordering only contention explains). Those numbers are discarded, not reported (Â§11.4.201(10)).

Instead the REGEX OPERATION COUNT was measured. A count is exact regardless of CPU contention, so it is valid under the conditions actually available (Â§11.4.201(8)). `re.sub/search/match/findall` were wrapped with a counter and `merge_results` run over synthetic result sets:

```
dataset A -- names drawn from a small vocabulary (many matches, groups col
     N    regex_ops    ops/N^2   growth per doubling
    50       40,256       16.1
   100      138,077       13.8   x3.4   (linear=x2, quadratic=x4)
   200      388,142        9.7   x2.8
   400    1,046,531        6.5   x2.7

dataset B -- highly distinct names (few matches, groups do not collapse)
     N    regex_ops    ops/N^2   growth per doubling
    50       20,395        8.2
   100       49,888        5.0   x2.4
```

```
200      129,764        3.2   x2.6
400      387,224        2.4   x3.0
```

Honest reading (Â§11.4.6): this is SUPER-LINEAR but, on these datasets, NOT yet fully quadratic â€” growth per doubling is 2.4â€“3.4 against 2.0 for linear and 4.0 for quadratic, and in dataset B it RISES with N (2.4 â† 2.6 â† 3.0). The claim supported by this measurement is â€œgrows substantially faster than the number of resultsâ€, not â€œmeasured NÂ²â€. The *worst case* is quadratic by code structure â€” `merge_results` (deduplicator.py:73-95) compares each seed against every remaining candidate, i.e.Â N(N-1)/2 comparisons when nothing matches â€” and dataset Bâ€™s rising trend is consistent with approaching that bound as N grows.

Per-comparison cost is the other half, and it is pure recomputation: `_normalize_name` is called at **four** sites per comparison (lines 275, 276 via `_compare_name_and_size`, and 357, 358 via `_compare_identities`), each running 5 `re.sub` passes; `_extract_identity_from_result` runs twice more (lines 224-225), each including `_detect_content_type`â€™s chain of up to ~15 sequential `re.search` calls. Nothing is memoised â€” `grep -n "lru_cache" merge_service/deduplicator.py` returns no matches â€” so the seedâ€™s own normalisation and identity are recomputed for every candidate it is compared against.

Scale actually observed on the live service: **572.7 s of continuous event-loop silence in a single episode**, and the loop was still inside `merge_results` more than 15 minutes after all load had stopped.

9. The instrument (`download-proxy/src/main.py`)

`py-spy` cannot attach on this host (`kernel.yama.ptrace_scope=1` denies non-child attach), so the process was made to dump its own stacks:

- an asyncio heartbeat task stamps a monotonic clock once a second from inside the merge-service loop;
- a plain watchdog thread notices when that stamp stops advancing and writes an all-thread traceback plus a per-thread CPU delta to `/config/download-proxy/diagnostics/stall_dumps.log` and to stderr;
- `SIGUSR1` (dump to log) and `SIGQUIT` (dump to file) give an on-demand dump with no ptrace: `podman exec qbittorrent-proxy kill -USR1 1`.

`faulthandler.dump_traceback()` is C-level and does not need the GIL, which is why it was used instead of `sys._current_frames()` â€” the latter needs the GIL and would hang in the same way as the thing it is measuring.

It adds **no** timeout, retry, restart or change to request handling.

The instrument is self-validated (Â§11.4.107(10), Â§11.4.201(7)(b))

A watchdog that never fires proves nothing â€” it could equally mean â€œno stallâ€ or â€œblind instrumentâ€ (Â§11.4.201(6) false-null). Both poles are asserted by `scripts/diagnostics/bob137_watchdog_selftest.py`, run against the containerâ€™s own Python 3.12:

```
== GOLDEN-BAD: event loop blocked -> watchdog MUST dump ==
   PASS: dumped and named the blocking frame
   PASS: per-thread CPU delta present (spin-vs-block discriminator)
== GOLDEN-GOOD: event loop healthy -> watchdog MUST stay quiet ==
   PASS: stayed quiet on a healthy loop

SELFTEST PASS - watchdog fires on a real stall and not otherwise.
```

Is it safe to leave in production?

Yes, with one disclosed behaviour change. Each property was checked, not assumed:

Concern	Finding
Steady-state cost	heartbeat wakes 1Ã—/s, watchdog thread wakes 1Ã—/2 s. No work while healthy.
Can it hang the thing it measures?	No.Â `faulthandler.dump_traceback()` is C-level and does not require the GIL.
Disk growth	Capped: `BOBA_STALL_MAX_DUMPS` (default 200) Ã— ~9.5 KB observed â‰ˆ 1.9 MB per process lifetime. Re-dump interval 60 s.
Repo pollution (Â§11.4.30)	None. Dumps land in `download-proxy/diagnostics/*.log`, already covered by `.gitignore:64` (`*.log`) â€” verified with `git check-ignore`.
Signal conflicts	None. No `SIGUSR1`/`SIGQUIT` handler exists in app source; `gunicorn` is not installed and `uvicorn.workers` (the only dependency referencing SIGQUIT) is never imported â€” verified in the live interpreter.
Shutdown path	Untouched. `SIGTERM`/`SIGINT` handlers are unchanged.
Failure mode	Fails open and loud: install is wrapped and logs `install FAILED`; internal dump errors are appended to `_diag_errors` and surfaced by the watchdog, never silently swallowed.
Kill switch	

Concern	Finding
	`BOBA_STALL_WATCHDOG=0` disables it entirely; thresholds tunable via env.
Â§11.4.263	The instrument sends no signals at all. It only *registers* handlers.

Disclosed behaviour change: `faulthandler.register(SIGQUIT, chain=False)` replaces SIGQUITâ€™s default disposition, so `kill -QUIT` now dumps stacks and the process CONTINUES instead of terminating with a core dump. For a service this is the safer disposition, but an operator relying on `kill -QUIT` to stop the container would find it ineffective â€” `SIGTERM` is unaffected and remains the correct stop signal. If that trade is unwanted, drop the SIGQUIT registration and keep SIGUSR1, which is unused by anything.

Recommendation: keep it. The defect is transient and self-clearing, so without an always-armed in-process observer the next occurrence is again only catchable by luck â€” which is how it escaped automated QA in the first place.

10. Discovery channel (Â§11.4.238)

Found by an agent hand-probing the host during an unrelated investigation â€” NOT by automated QA. Coverage escape. Two escapes are now visible:

1. the health check could not observe this (sibling item BOB-138);
2. no automated check exercises concurrent search fan-out and asserts the merge service stays responsive during it â€” which is precisely what `bob137_soak.sh` does, and it reproduced the defect in under two minutes.

11. Artifacts

File	Contents
`stall_dump_1.txt`	7 all-thread stack dumps + per-thread CPU deltas
`repro_socket_state.txt`	live socket/thread state during the reproduced wedge
`soak_probe.log`	per-probe 7186/7187 latency and HTTP code
`soak_driver.log`	fan-out request outcomes
`sockets.txt`, `threads.txt`, `container_log_tail.txt`	Revision 1 captures
`scripts/diagnostics/bob137_watchdog_selftest.py`	golden-good/golden-bad self-test
`scripts/diagnostics/bob137_soak.sh`	soak reproducer